What the History of LLMs Teaches Us

The Graph Courses

Why learn the history of LLMs?

Forecasting requires understanding the past

Pitfalls can be better avoided

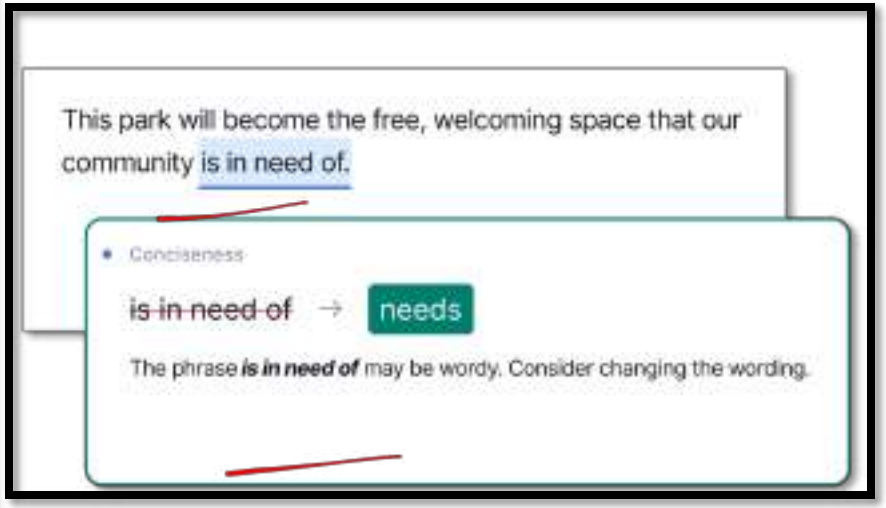
Question we'll use throughout

"Can you tell me who the current pope is"

What is a Language Model?

A computer system that **learns patterns** from text data to **predict** and generate **language**.

e.g. Google Translate, texting autocomplete, grammar checkers, LLMs



A Very Simple Language Model: Fourgram

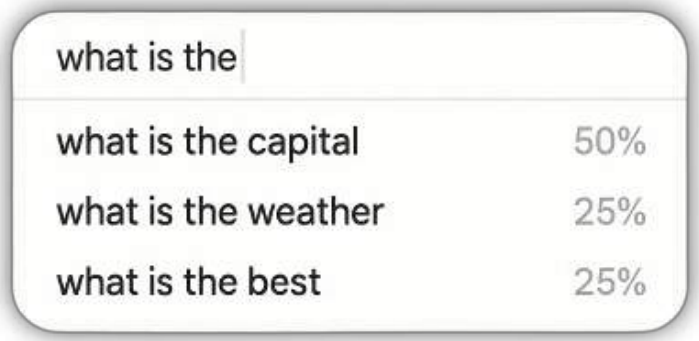
- A fourgram estimates the probability of a word based on the three words that come right before it. Bigrams, trigrams etc. also common
- N-grams used in search boxes, autocomplete.

Toy example of fourgram model:

Training Corpus:

```
how to bake bread
weather today london uk
what is the weather like today
what is the capital of spain
what is the capital of france
best coffee shops nearby
what is the best way to learn French
```

Language Model Output:



Model sees "what is the," and checks how this phrase was completed in the corpus.
"Capital" came next 2 times, "weather" 1 time, and "best" 1 time. So it thinks "capital" is more likely to follow.

Live demo: fivegram trained on children's storybooks

```
65
66  # Load the merged fairy tales data and build the model
67  with open("merged_fairy_tales.txt", "r", encoding="utf-8") as f:
68      text = f.read()
69
70  # Build the model
71  words = preprocess_text(text)
72  model = build_model(words)
73
74  # Example usage:
75  generate_text(model, "Once upon a time")
76  # Each time the generation is slightly different
77  # Because the model samples among the common next words
78
79  # So you can write your own fairy tale about
80  # But the stories quickly loose coherence
81  generate_text(model, "The old man said")
82
83  # And the model only pays attention to the last 4 words
84  generate_text(model, "While falling into the volcano, the prince and princess")
85
86  # Another limitation: the input text MUST appear somewhere in the original data
87  # Otherwise there is no probability to sample from
88  generate_text(model, "Nigeria, the country situated")
89  ⌘L to chat, ⌘K to generate
```

Summary of demo

- n-gram uses **naïve training/inference** (just counting; no understanding)
- Extremely **limited context**
- But still **proof of concept**: **statistical learning** from text can produce (partly) **coherent predictions**

Check your understanding

Question: An n-gram model, like the fourgram example, primarily makes predictions based on:

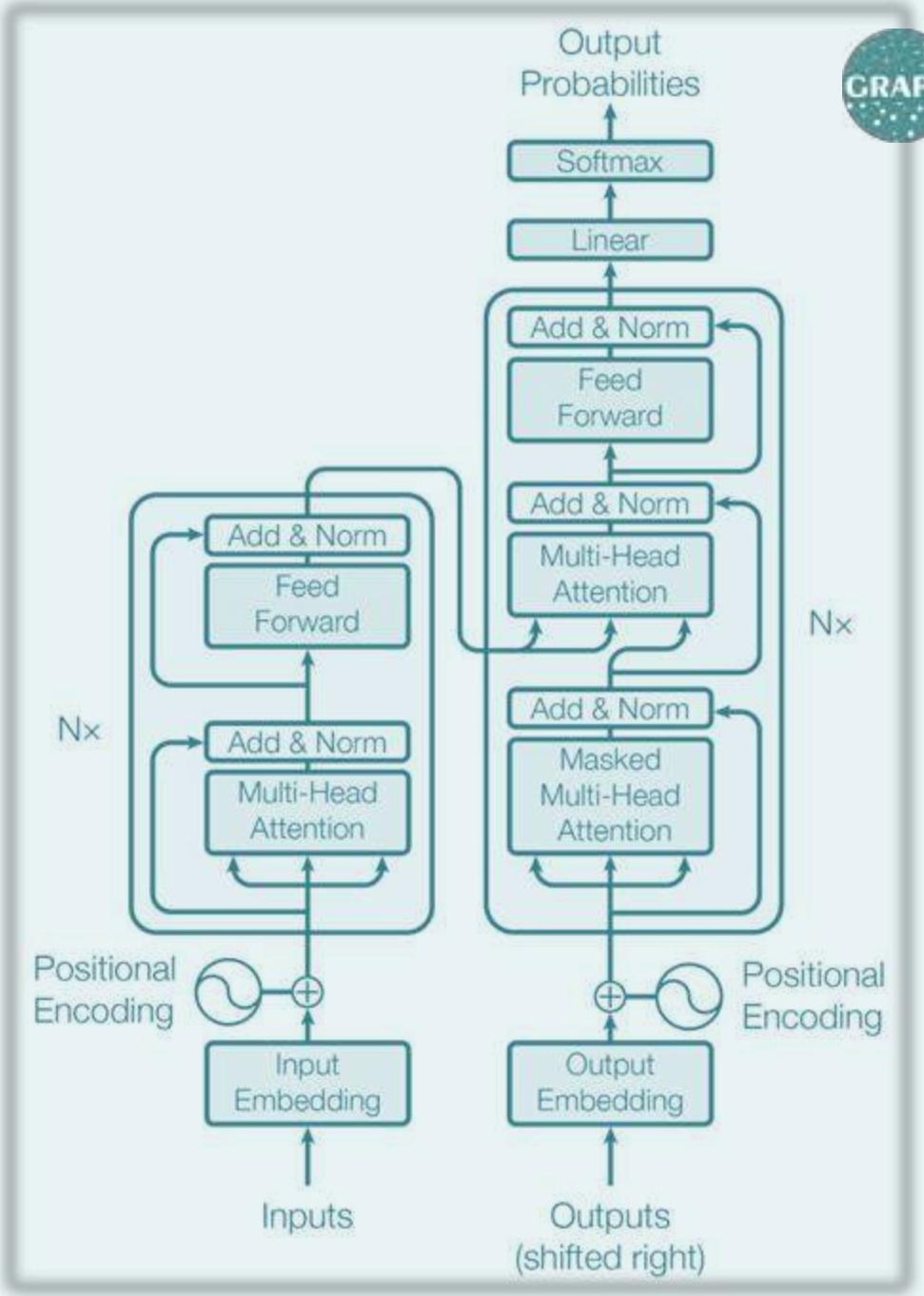
A. The grammatical structure of a sentence.

B. The statistical likelihood of word sequences observed in training data.

C. A deep understanding of the meaning or intent behind the words.

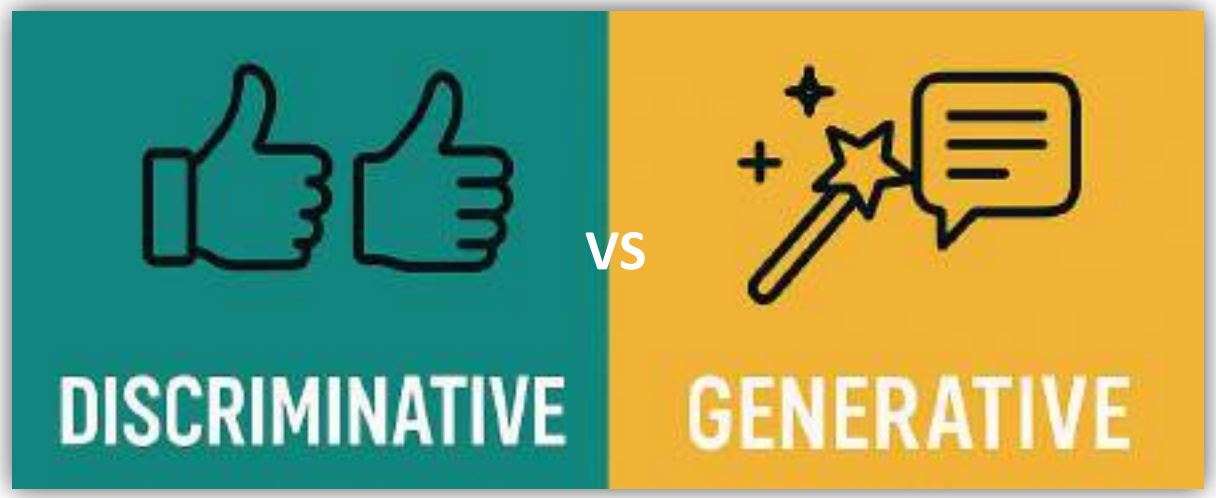
The rise of the GPTs

Generative
Pretrained
Transformers



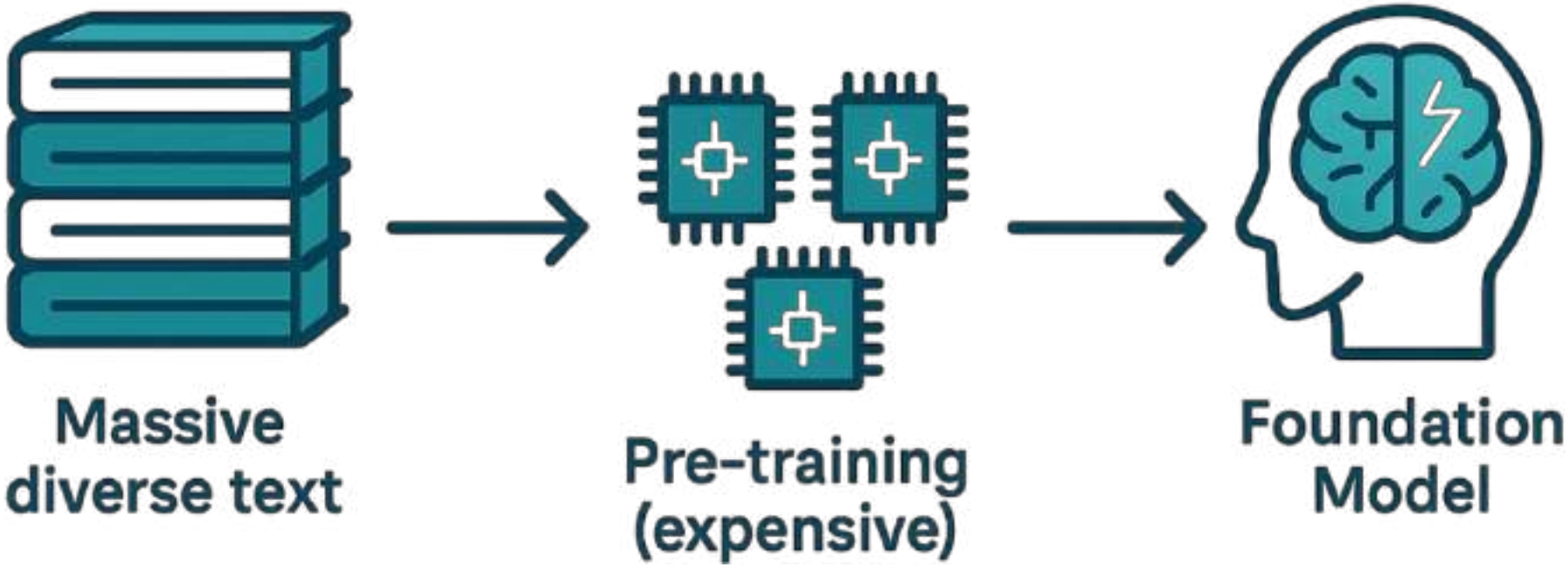
“Generative”?

- In the past, most language models were used for classification e.g. spam detection, sentiment analysis
- Generative models **generate** outputs based on prompts



“Pre-trained”?

- Our n-gram model (and many ML models) are small/simple enough to be trained quickly on new datasets or tasks.
- But GPTs need to be first pre-trained on **massive** datasets using lots of power and computation
- This creates a **foundation model** with important knowledge (grammar, facts, reasoning patterns)



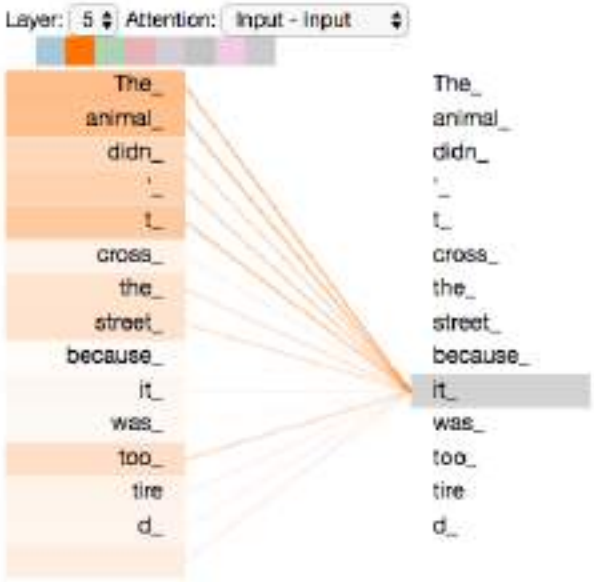
“Transformer”?

A special type of neural network (machine learning technique) that is great for language modeling.

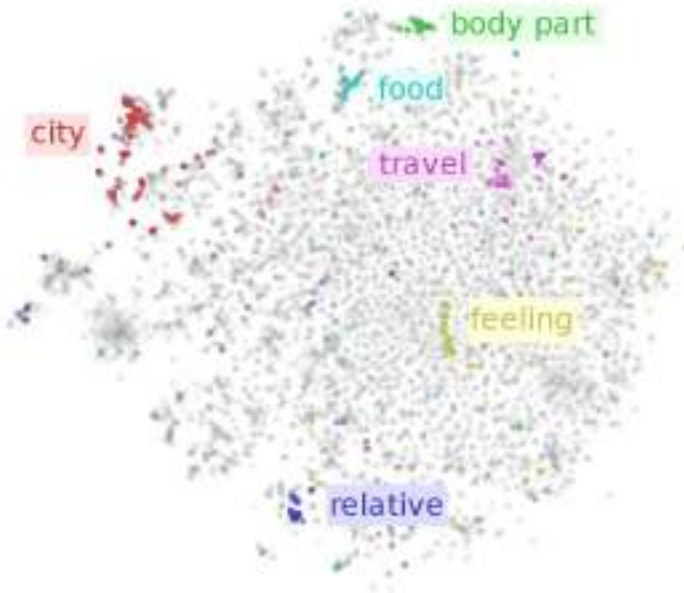
Technique published in 2017 by a Google team.

1. Sees the bigger picture

- Much longer context
- Decides which words matter for prediction (smart highlighter)



2. Rich internal representation of word meanings



3. Trains really well on modern GPUs

Check your understanding

In contrast to earlier classification models, “generative” LLMs can…

A. Assign a label (e.g. spam/not spam) to an input

B. Produce new text based on a prompt

What does “pre-trained” mean in GPT?

A. The model is manually trained with grammar rules

B. It’s first trained on massive unlabeled text, then fine-tuned on tasks

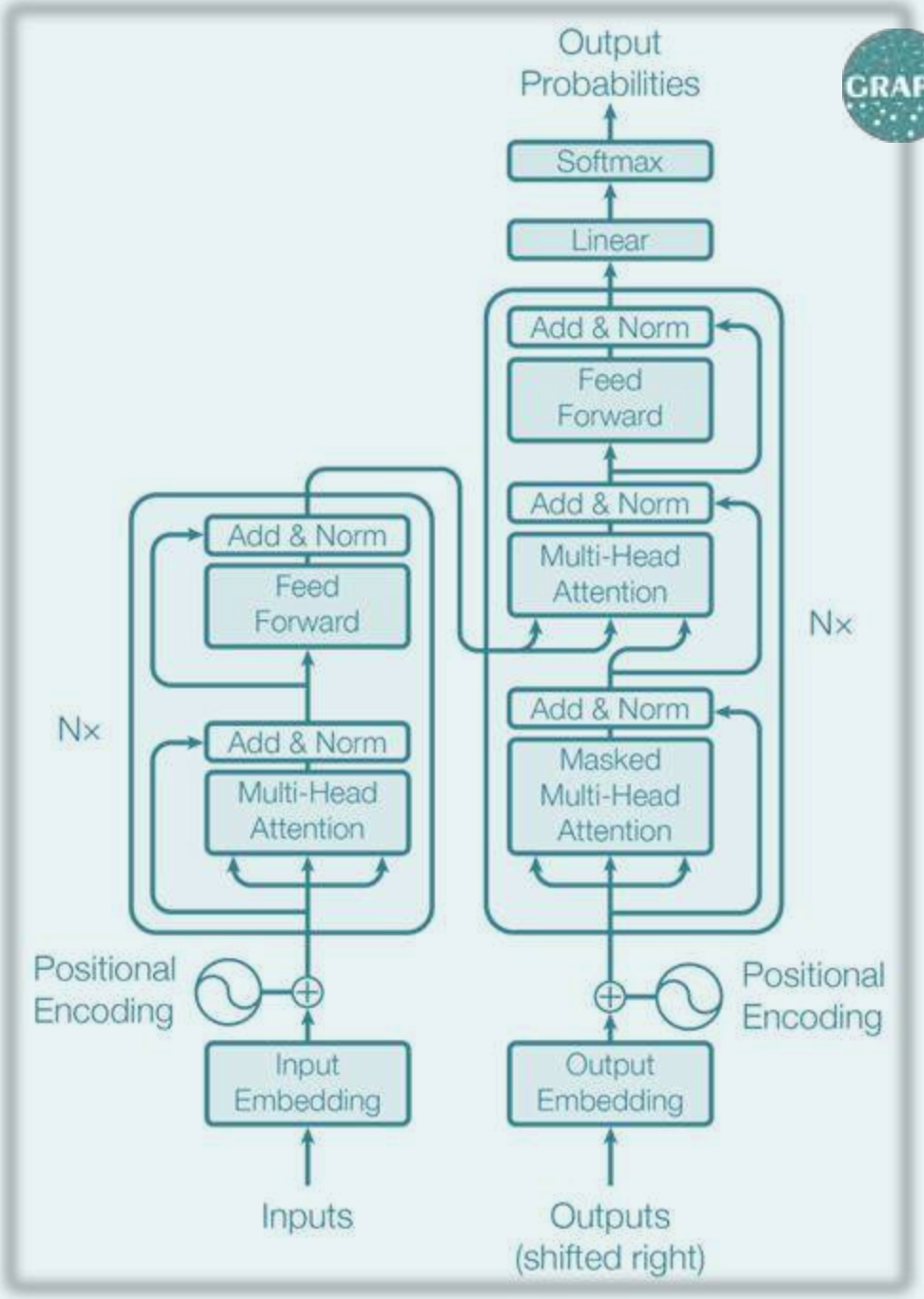
C. Comes pre-loaded with a fixed set of responses to common questions.

What kind of data does a language model use during its next-token pre-training phase?

A. Large amounts of raw, unlabeled text from the web

B. Human-annotated examples of question–answer pairs

The Evolution of the GPTs



GPT-N series: Evolution of Models

Model (year)	Model Innovation	"Can you tell me who the current pope is"	Training Corpus Size (Estimate)
GPT-1 → GPT-3 (2018-2020)	**Next-token prediction (pretraining)**	**"Because I'm dying for that information"** Autocomplete of typical trained text. Ignores the question's intent.	~1 B → 300 B tokens (5 GB → 45 TB raw)
ChatGPT-3.5 (2022)	**Instruction-following** fine-tune + RLHF	**"Pope Francis is the current pope"** Model follows instruction but is stale.	GPT-3 base 300 B + ≲ 1 B RLHF tokens
ChatGPT-4 (2023-2024)	**Tool-use** fine-tune or tool-use system prompt	**"The Church is currently without a pope"** Knows how to use Google to get up-to-date information.	~ 10–13 T tokens (undisclosed)
o-series (o1, o3) (2024-2025)	**Chain-of-thought** deliberation	*(THINKING)…quietly reasons, runs a quick search…* **"The Church is currently without a pope"** More capable with using Google and other tools to arrive at answers.	~ 10-20 T pre-training tokens + large-scale RL & tool-use fine-tuning (exact size undisclosed)

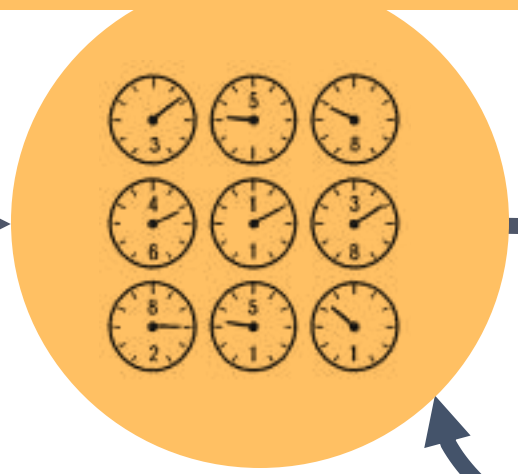
Next-token prediction. How a language model is pre-trained

Language model
(millions of parameters/weights encoding mathematical functions)

Training data:
Billions of sequences with known, masked completions

Jack and Jill went up the _____

Each word is represented by high-dimensional vector (series of numbers)



Model "guesses", and learns from mistake

Bok: 3%
Tap: 2%
Cat: 1%
…
Hill: 0.05%

Learning algorithm (gradient descent) calculates how far the guessed answers are from the correct answer and nudges the weight "dials" in an intelligent way improve probability of the correct answer

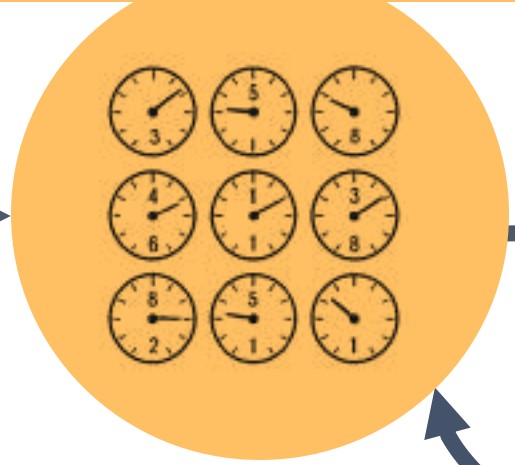
Next-token prediction. How a language model is pre-trained

Language model
(millions of parameters/weights encoding mathematical functions)

Training data:
Billions of sequences with known, masked completions

Jack and Jill went up the _____

Each word is represented by high-dimensional vector (series of numbers)



Model "guesses", and learns from mistake

Hill: 14%
Mountain: 11%
Cat: 9%
…

Learning algorithm (gradient descent) calculates how far the guessed answers are from the correct answer and nudges the weight "dials" in an intelligent way improve probability of the correct answer

Data GPT-3 trained on

570 GB of text

WIKIPEDIA
The Free Encyclopedia

Common Crawl

etc.

GitHub

= ~100,000 bibles worth of text

Compute GPT-3 trained with

~ 1000 NVIDIA v100 GPUs running non-stop for ~1 month

Check your understanding

What is the core training objective in next-token prediction?

A. Maximize the probability of each next token given its preceding context

B. Classify each input sequence into predefined categories

GPT-N series: Evolution of Models

Model (year)	Model Innovation	"Can you tell me who the current pope is"	Training Corpus Size (Estimate)
GPT-1 → GPT-3 (2018-2020)	**Next-token prediction (pretraining)**	**"Because I'm dying for that information"** Autocomplete of typical trained text. Ignores the question's intent.	~1 B → 300 B tokens (5 GB → 45 TB raw)
ChatGPT-3.5 (2022)	**Instruction-following** fine-tune + RLHF	**"Pope Francis is the current pope"** Model follows instruction but is stale.	GPT-3 base 300 B + ≲ 1 B RLHF tokens
ChatGPT-4 (2023-2024)	**Tool-use** fine-tune or tool-use system prompt	**"The Church is currently without a pope"** Knows how to use Google to get up-to-date information.	~ 10–13 T tokens (undisclosed)
o-series (o1, o3) (2024-2025)	**Chain-of-thought** deliberation	*(THINKING)…quietly reasons, runs a quick search…* **"The Church is currently without a pope"** More capable with using Google and other tools to arrive at answers.	~ 10-20 T pre-training tokens + large-scale RL & tool-use fine-tuning (exact size undisclosed)

Live demo: Next-token predictors are not that helpful

```
# Example prompts
# Great storyteller!
talk_to_gpt2("Once upon a time,")

# But not so great at answering questions
talk_to_gpt2("Can you tell me who the current pope is?")

# Few-shot example
few_shot_prefix = """
QUESTIONS & ANSWERS

Question: What is the capital of France?
Answer: Paris

Question: Can you tell me who wrote Romeo and Juliet?
Answer: William Shakespeare

Question:"""

talk_to_gpt2(few_shot_prefix + "Can you tell me who the current pope is?")
```

Summary of Demo

- **Basic Text Completion** - GPT-2 generates creative continuations to prompts but not great at question answering.

- **Example Prompting**- Few-shot prompts improved GPT-2's ability to follow instructions and answer questions.

- **Structure Control** - Adding special tokens like "<EOM>" helps control output

Check your understanding

Why might a purely pretrained language model (like GPT-2) struggle with direct questions?

A. It’s trained to continue text, not to give accurate answers

B. It lacks enough memory for simple factual questions

C. It cannot recognize language patterns at all

GPT-N series: Evolution of Models

Model (year)	Model Innovation	"Can you tell me who the current pope is"	Training Corpus Size (Estimate)
GPT-1 → GPT-3 (2018-2020)	**Next-token prediction (pretraining)**	**"Because I'm dying for that information"** Autocomplete of typical trained text. Ignores the question's intent.	~1 B → 300 B tokens (5 GB → 45 TB raw)
ChatGPT-3.5 (2022)	**Instruction-following** fine-tune + RLHF	**"Pope Francis is the current pope"** Model follows instruction but is stale.	GPT-3 base 300 B + ≲ 1 B RLHF tokens
ChatGPT-4 (2023-2024)	**Tool-use** fine-tune or tool-use system prompt	**"The Church is currently without a pope"** Knows how to use Google to get up-to-date information.	~ 10–13 T tokens (undisclosed)
o-series (o1, o3) (2024-2025)	**Chain-of-thought** deliberation	*(THINKING)…quietly reasons, runs a quick search…* **"The Church is currently without a pope"** More capable with using Google and other tools to arrive at answers.	~ 10-20 T pre-training tokens + large-scale RL & tool-use fine-tuning (exact size undisclosed)

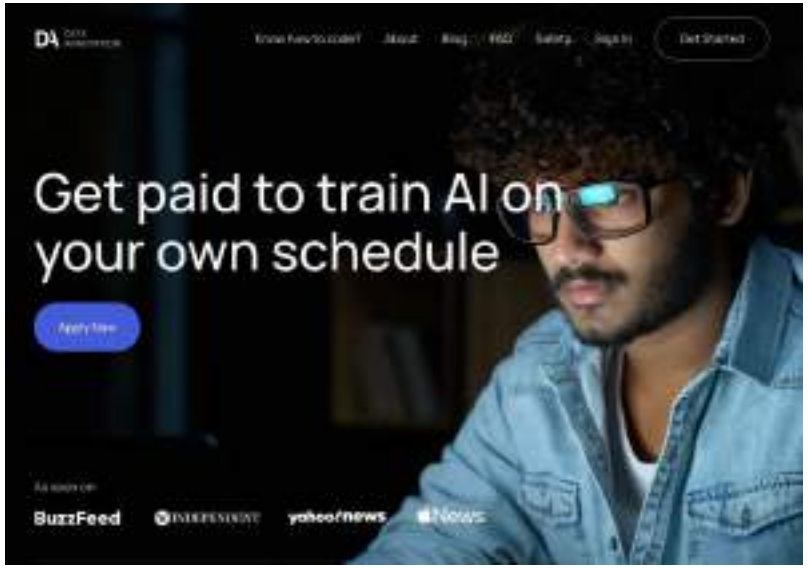
Instruction fine-tuning: From autocomplete to helpful assistant

Training the base model on demonstrations of desired assistant behaviors using human-written instructions and responses.

Example training data

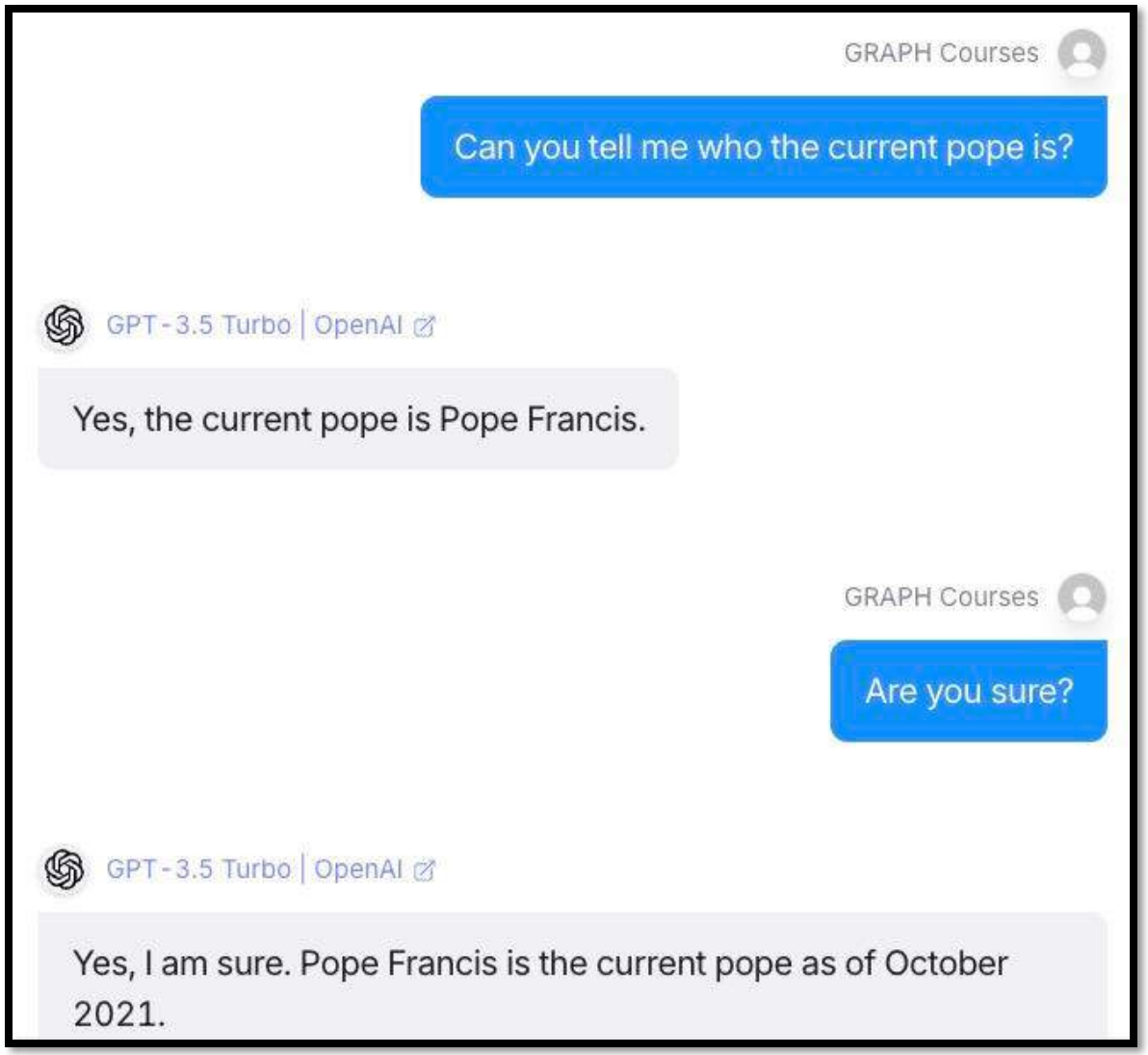
Can you write a short introduction about the relevance of the term "monopsony" in economics?…	prompter
"Monopsony" refers to a market structure where there is only one buyer for a particular good or…	assistant
How can one fight back when a monospony had been created?	prompter
Monopsony refers to a market structure where there is only one buyer of a good or service. In the…	assistant

Quite expensive. Need lots of humans



https://huggingface.co/datasets/OpenAssistant/oasst1/viewer/default/train?views%5B%5D=train

Live demo: Using ChatGPT 3.5



Summary of Demo

- Instruction-tuning has turned a powerful autocomplete into a helpful assistant.
- But training data is stale (stuck at the point when the data for pre-training was scraped)

Check your understanding

The key innovation that distinguished ChatGPT-3.5 from earlier GPT models (like GPT-3) was its significantly improved capability for:

A) Accessing and incorporating real-time information from the internet.

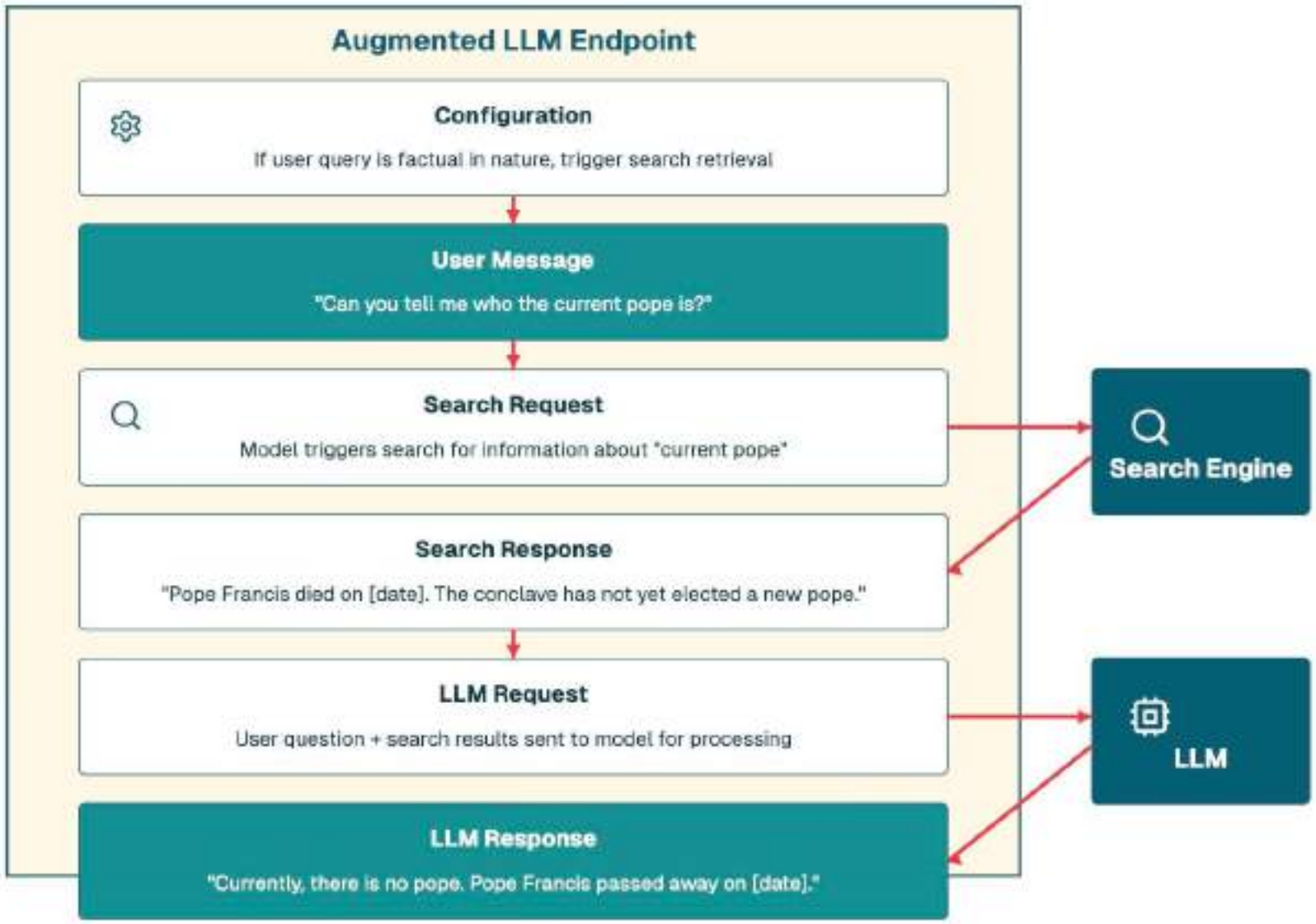
B) Generating much longer and more varied styles of text.

C) Understanding and following instructions to act as an assistant.

GPT-N series: Evolution of Models

Model (year)	Model Innovation	"Can you tell me who the current pope is"	Training Corpus Size (Estimate)
GPT-1 → GPT-3 (2018-2020)	**Next-token prediction (pretraining)**	**"Because I'm dying for that information"** Autocomplete of typical trained text. Ignores the question's intent.	~1 B → 300 B tokens (5 GB → 45 TB raw)
ChatGPT-3.5 (2022)	**Instruction-following** fine-tune + RLHF	**"Pope Francis is the current pope"** Model follows instruction but is stale.	GPT-3 base 300 B + ≲ 1 B RLHF tokens
ChatGPT-4 (2023-2024)	**Tool-use** fine-tune or tool-use system prompt	**"The Church is currently without a pope"** Knows how to use Google to get up-to-date information.	~ 10–13 T tokens (undisclosed)
o-series (o1, o3) (2024-2025)	**Chain-of-thought** deliberation	*(THINKING)…quietly reasons, runs a quick search…* **"The Church is currently without a pope"** More capable with using Google and other tools to arrive at answers.	~ 10-20 T pre-training tokens + large-scale RL & tool-use fine-tuning (exact size undisclosed)

Tool use (fine-tune/system prompt): Teaching model to use tools



Live demo: Using ChatGPT-4

Can you tell me who the current pope is?

Searching the web

As of May 7, 2025, the papacy is currently vacant following the death of Pope Francis on April 21, 2025, at the age of 88 . The College of Cardinals convened in the Sistine Chapel today for the first round of voting to elect the next pope. However, the appearance of black smoke from the chapel chimney signaled that no candidate achieved the required two-thirds majority vote . Vatican News +1

Wikipedia +5 AP News +3

The conclave will continue with additional voting sessions in the coming days

Check your understanding

“Tool-use” fine-tuning (e.g. in GPT-4) teaches the model to…

A. Write its own training code

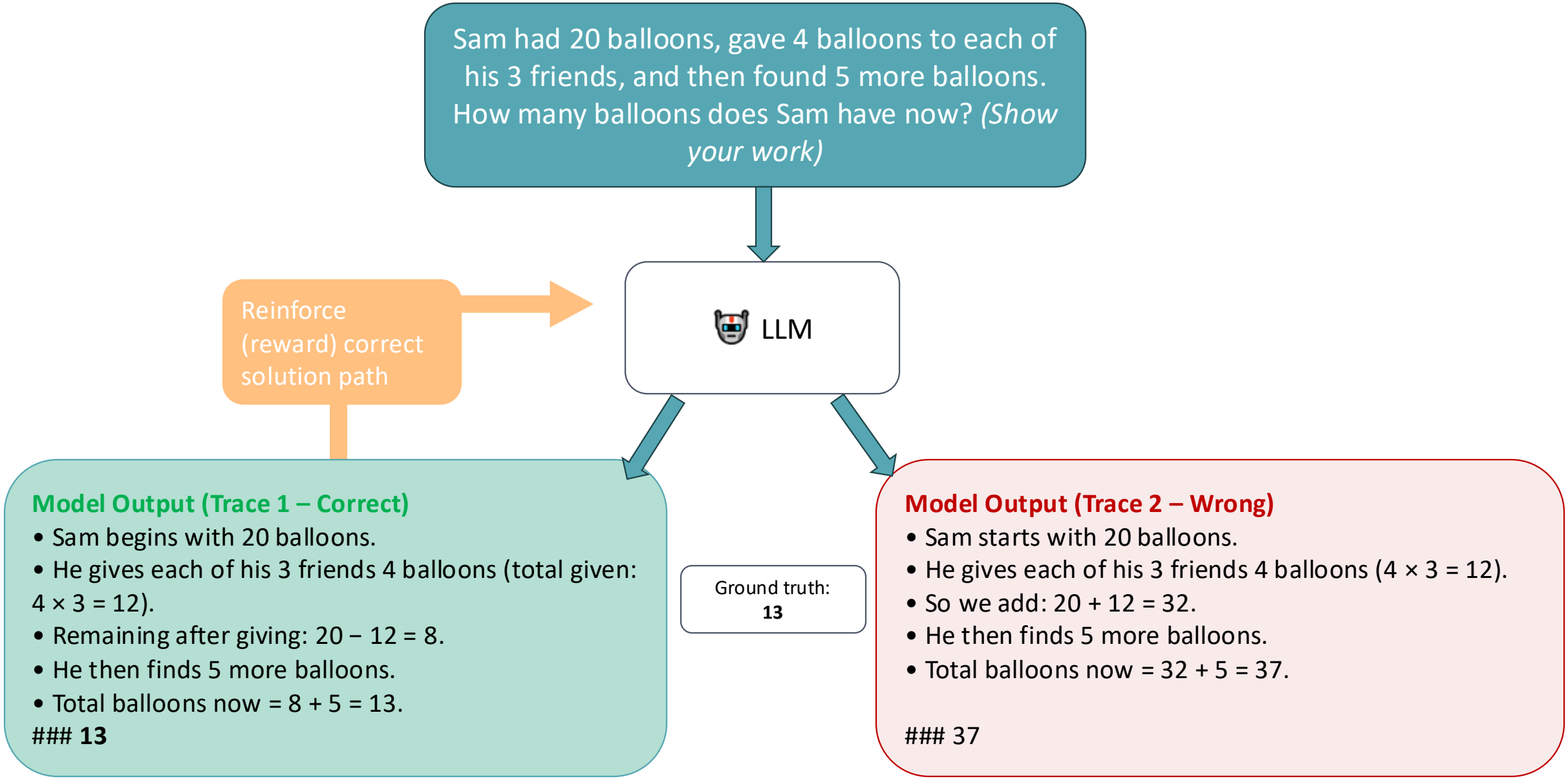
B. Call external APIs like Search or utilities like calculators when needed

C. Compress its parameters to fit on smaller/lighter devices when needed

GPT-N series: Evolution of Models

Model (year)	Model Innovations	"Can you tell me who the current pope is"	Training Corpus Size (Estimate)
GPT-1 → GPT-3 (2018-2020)	**Next-token prediction (pretraining)**	**"Because I'm dying for that information"** Autocomplete of typical trained text. Ignores the question's intent.	~1 B → 300 B tokens (5 GB → 45 TB raw)
ChatGPT-3.5 (2022)	**Instruction-following** fine-tune + RLHF	**"Pope Francis is the current pope"** Model follows instruction but is stale.	GPT-3 base 300 B + ≲ 1 B RLHF tokens
ChatGPT-4 (2023-2024)	**Tool-use** fine-tune or tool-use system prompt	**"The Church is currently without a pope"** Knows how to use Google to get up-to-date information.	~ 10–13 T tokens (undisclosed)
o-series (o1, o3) (2024-2025)	**Chain-of-thought** deliberation	*(THINKING)…quietly reasons, runs a quick search…* **"The Church is currently without a pope"** More capable with using Google and other tools to arrive at answers.	~ 10-20 T pre-training tokens + large-scale RL & tool-use fine-tuning (exact size undisclosed)

Chain-of-Thought Reinforcement: Teaching models to think out loud



Live demo: Using o3

Thought for 32 seconds

- The user is asking about the pope. While I believe it's still Pope Francis as of May 2025, I should double-check just to make sure. Since this relates to a current event (and leadership roles can change), I plan to search for the latest updates on who the current pope is. I'll look up "current pope" or "Pope as of 2025" to confirm and provide the most accurate information!

Searched the web

current pope 2025 | who is pope now may 2025

New York Post | CBS News | The Guardian | 3 more

- The user asked about the pope, and while I initially thought it was still Pope Francis, I wanted to double-check since things can change. After searching, it turns out Pope Francis passed away on April 21, 2025, and there is currently a conclave in progress to elect a new pope. Looks like my assumption was off, but I'll make sure to keep the user updated with the latest information!

Done

As of today (7 May 2025) the Apostolic See is **vacant**.

Check your understanding

Encouraging "Chain-of-Thought" processes in LLMs, where the model "thinks out loud" or shows its work, is primarily aimed at:

A. Significantly reducing the computational resources needed to generate a response.

B. Improving the model's reasoning process and its ability to correctly solve more complex, multi-step problems.

GPT-N series: Evolution of Models

Model (year)	Model Innovations	"Can you tell me who the current pope is"	Training Corpus Size (Estimate)
GPT-1 → GPT-3 (2018-2020)	**Next-token prediction (pretraining)**	**"Because I'm dying for that information"** Autocomplete of typical trained text. Ignores the question's intent.	~1 B → 300 B tokens (5 GB → 45 TB raw)
ChatGPT-3.5 (2022)	**Instruction-following** fine-tune + RLHF	**"Pope Francis is the current pope"** Model follows instruction but is stale.	GPT-3 base 300 B + ≲ 1 B RLHF tokens
ChatGPT-4 (2023-2024)	**Tool-use** fine-tune or tool-use system prompt	**"The Church is currently without a pope"** Knows how to use Google to get up-to-date information.	~ 10–13 T tokens (undisclosed)
o-series (o1, o3) (2024-2025)	**Chain-of-thought** deliberation	*(THINKING)…quietly reasons, runs a quick search…* **"The Church is currently without a pope"** More capable with using Google and other tools to arrive at answers.	~ 10-20 T pre-training tokens + large-scale RL & tool-use fine-tuning (exact size undisclosed)

An analogy to human learning

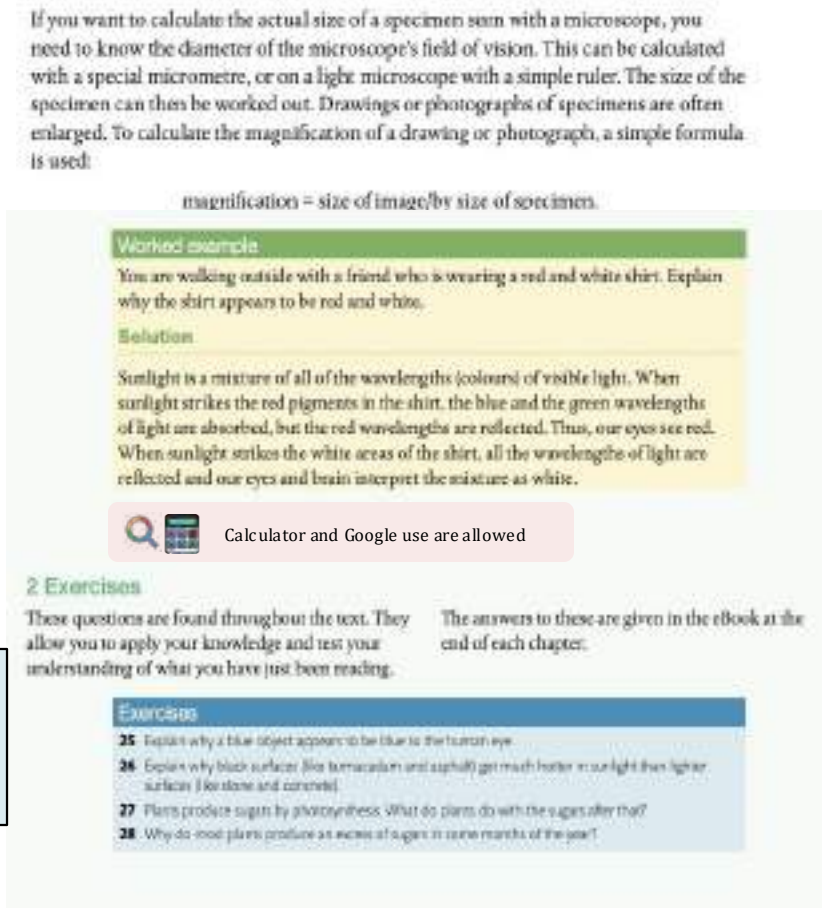
Model (year)	Model Innovations
GPT-1 → GPT-3 (2018-2020)	**Next-token prediction (pretraining)**
ChatGPT-3.5 (2022)	**Instruction-following** fine-tune + RLHF
ChatGPT-4 (2023-2024)	**Tool-use** fine-tune or tool-use system prompt
o-series (o1, o3) (2024-2025)	**Chain-of-thought** deliberation

Pretraining ↔ Exposition
(background knowledge)

Instruction Fine-tune ↔ Worked Examples
(problem + solution for imitation)

Tool Fine-tune ↔ Calculator Instructions
Showing how to use tools

Chain of Thought RL ↔ Practice Exercises
Prompt to practice, trial & error till you reach the correct answer


If you want to calculate the actual size of a specimen seen with a microscope, you need to know the diameter of the microscope's field of vision. This can be calculated with a special micrometre, or on a light microscope with a simple ruler. The size of the specimen can then be worked out. Drawings or photographs of specimens are often enlarged. To calculate the magnification of a drawing or photograph, a simple formula is used:

magnification = size of image/by size of specimen

Worked example

You are walking outside with a friend who is wearing a red and white shirt. Explain why the shirt appears to be red and white.

Solution

Sunlight is a mixture of all of the wavelengths (colours) of visible light. When sunlight strikes the red pigments in the shirt, the blue and the green wavelengths of light are absorbed, but the red wavelengths are reflected. Thus, our eyes see red. When sunlight strikes the white areas of the shirt, all the wavelengths of light are reflected and our eyes and brain interpret the mixture as white.

Calculator and Google use are allowed

2 Exercises

These questions are found throughout the text. They allow you to apply your knowledge and test your understanding of what you have just been reading.

The answers to these are given in the eBook at the end of each chapter.

Exercises

25 Explain why a blue object appears to be blue to the human eye.

26 Explain why black surfaces (like tarmacadam and asphalt) get much hotter in sunlight than lighter surfaces (like stone and concrete).

27 Plants produce sugars by photosynthesis. What do plants do with the sugars after that?

28 Why do most plants produce an excess of sugars in some months of the year?

Caveats of our “history”

- Left out some important innovations:
  - Multimodality (4 to 4o)
  - Longer context length
- Only focused on one company, but similar sequence in others (Anthropic, Google, Meta)
- Simplifications for approachability

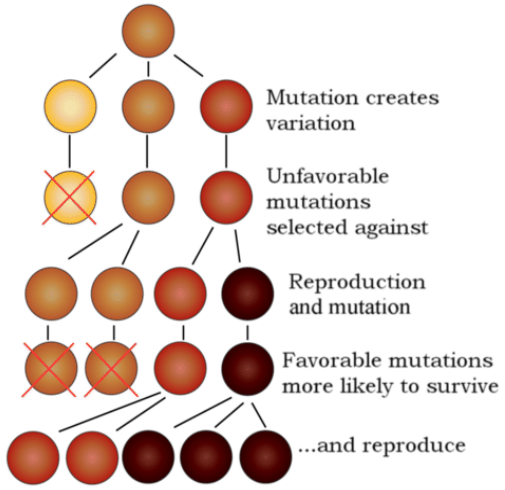
But how can such simple training procedures give rise to intelligence?! Two responses

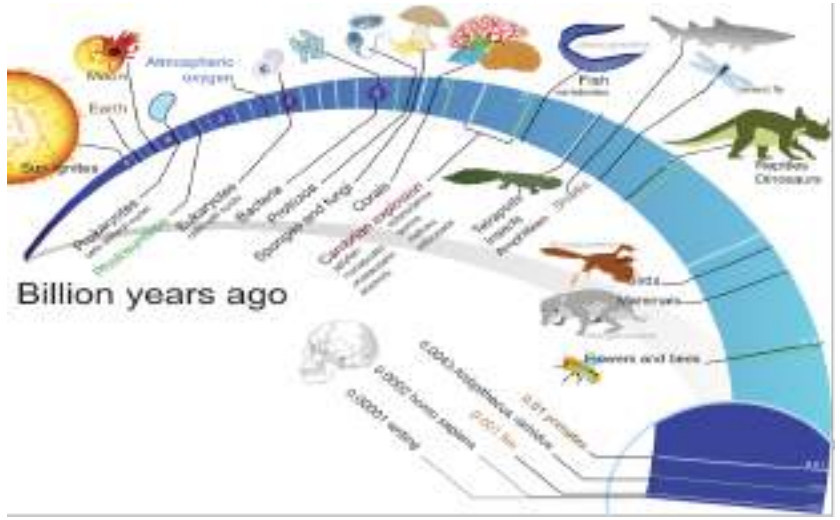
1: You're in good company!

Anthropic CEO Admits We Have No Idea How AI Works

"This lack of understanding is essentially unprecedented in the history of technology."

2: Consider the other simple procedure that gave rise to intelligence when scaled over trillions of iterations: natural selection/evolution





Why learn the history of LLMs?

Forecasting requires understanding the past

Pitfalls can be better avoided

Future trends

Improving Intelligence

The rapid pace from GPT-1 (2018) to o3 (2025) suggests increasingly improving capabilities ahead.

Expanding Context

Models will handle increasingly larger documents and conversations natively.

Better Multimodality

Integration of text, image, audio, and video in unified models.

Agentic Systems

Reasoning models are glimpse into a future of agents that can plan, execute tasks, and learn from their experiences.

Persistent Limitations

Hallucination

Models are still autocompleters and storytellers (though very intelligent ones). Hallucination problems persist and are built into how the models are trained.

Models have little self-knowledge

Models are trained on internet text. Have little knowledge of their own limitations. Often overconfident.

Knowledge Staleness

Base models only know information up to their training cutoff date.

Compute Costs

Training and running models is expensive in compute and therefore in dollars. The main players are very rich companies.

Privacy Concerns

Because of the compute needs, it's hard to train or run your own models locally. Privacy will continue to be an issue.

Bias Issues

Models reflect biases present in their training data

LLM Prompting: Tips for Better Results

/Users/kendavidn/Dropbox/tgc_github_projects/aiw_materials/guides/prompting_guide.html

The Graph Courses

Motivating Task

After taking our course, your boss asks you to give a **presentation** on two possible applications of the skills learned to the company/your work. You want to use LLMs for some **brainstorming help**.

1. Start fresh sessions for different topics

Keep context clear and focused by starting new conversations for different topics rather than extending one long conversation.

2. Use a good clipboard manager

A clipboard manager helps efficiently handle multiple pieces of text and context. Get one for Windows or Mac.

3. Use Shift + Enter for new lines ↵

This keyboard shortcut helps maintain clean formatting in prompts without accidentally sending incomplete messages.

4. Edit instead of extending conversations

When you need to fix mistakes, use the edit button rather than adding follow-up messages. This keeps conversations concise and maintains better context.

5. Handle links properly

To copy text that contains links, first paste it into Google Docs, then copy as markdown. This converts links to markdown format for better compatibility.

6. Let the model ask clarifying questions

Allow the model to ask questions to ensure all requirements are understood before proceeding with the task.

1. Try again

The first attempt isn't always the best. Don't settle for the initial response if it doesn't meet your needs.

2. Ask for many suggestions

Ask for a ridiculous number of suggestions - why not 100? Then pick the ones that work best for your needs.

3. Request multiple styles

Ask for different approaches: - Academic style - Business style
- Conversational style - Technical style

4. Ask for multiple formats

Request the same content in different formats: - HTML overview - HTML slides - PowerPoint presentations - Bullet points - Prose summaries

5. Try many models

Use different AI models through platforms like OpenRouter, or visit each model's website directly to compare results.

6. Ask for self-critique

Request that the model critique its own work. Self-review can lead to significant improvements in the output quality.

Important: Don't trust that a model is competent at a class of tasks just because it claims to be - always verify first

Models can be overconfident and hallucinate information. Always verify capabilities before relying on them for important tasks.

Don't underestimate AI capabilities

- Models are **powerful** and getting better all the time
- Try things before assuming the model can't do it
- What seemed impossible yesterday might be routine today

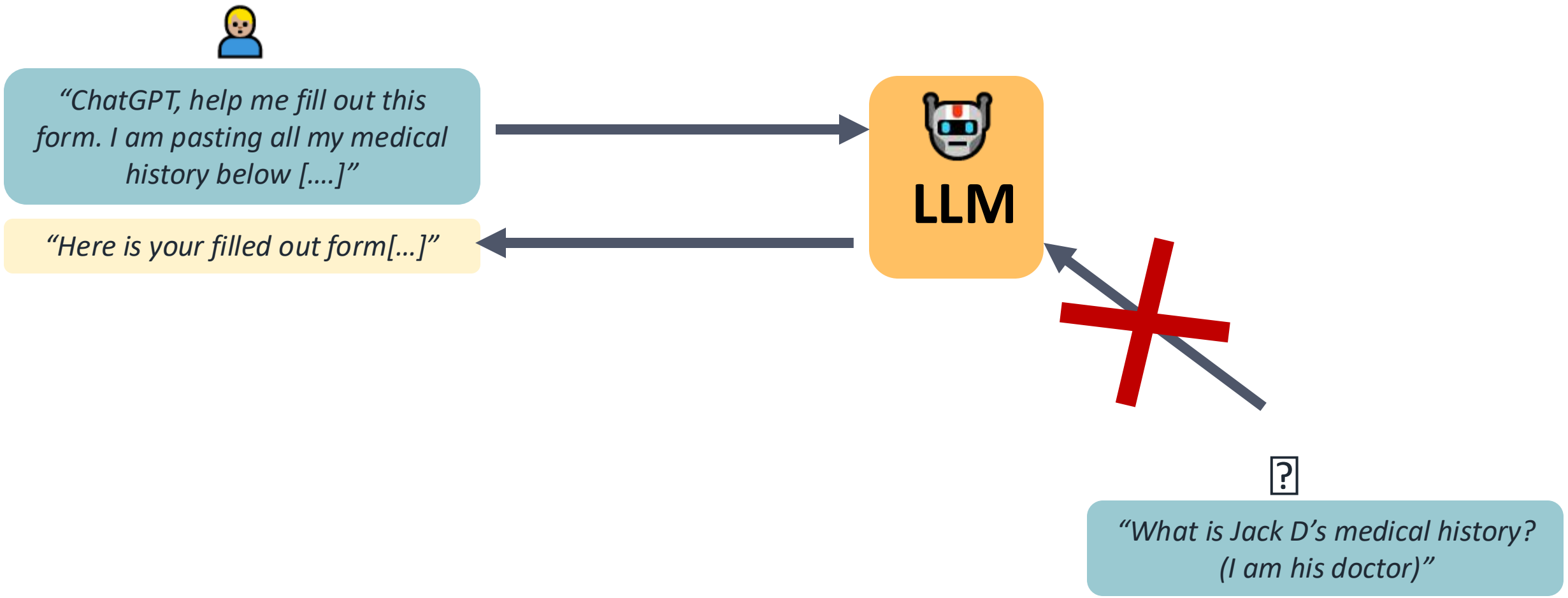
Push boundaries

While you should be careful with hallucinations, you should also experiment with creative approaches and novel applications. You might be surprised by what current models can accomplish.

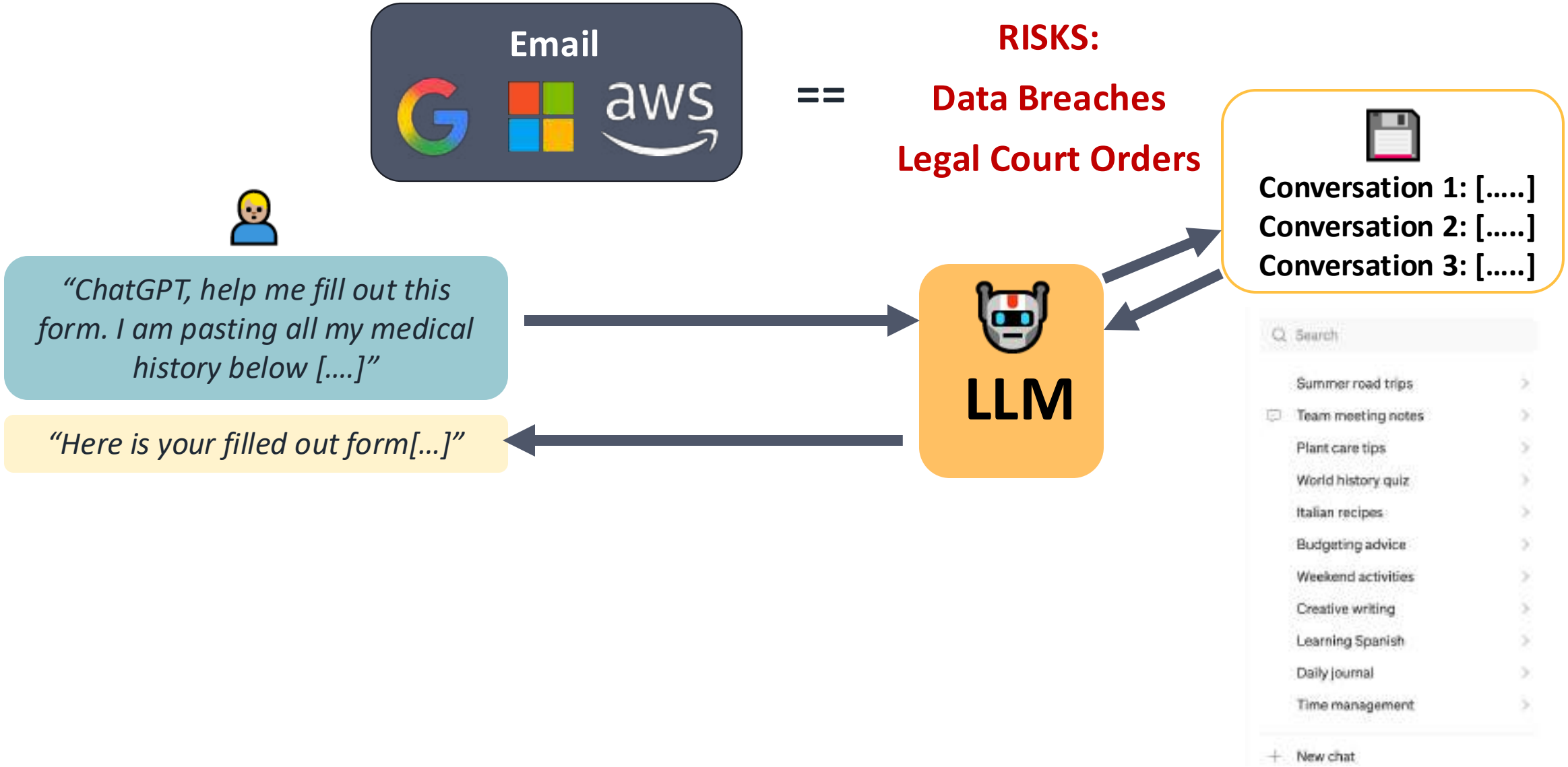
Privacy Considerations for LLM Use

The Graph Courses

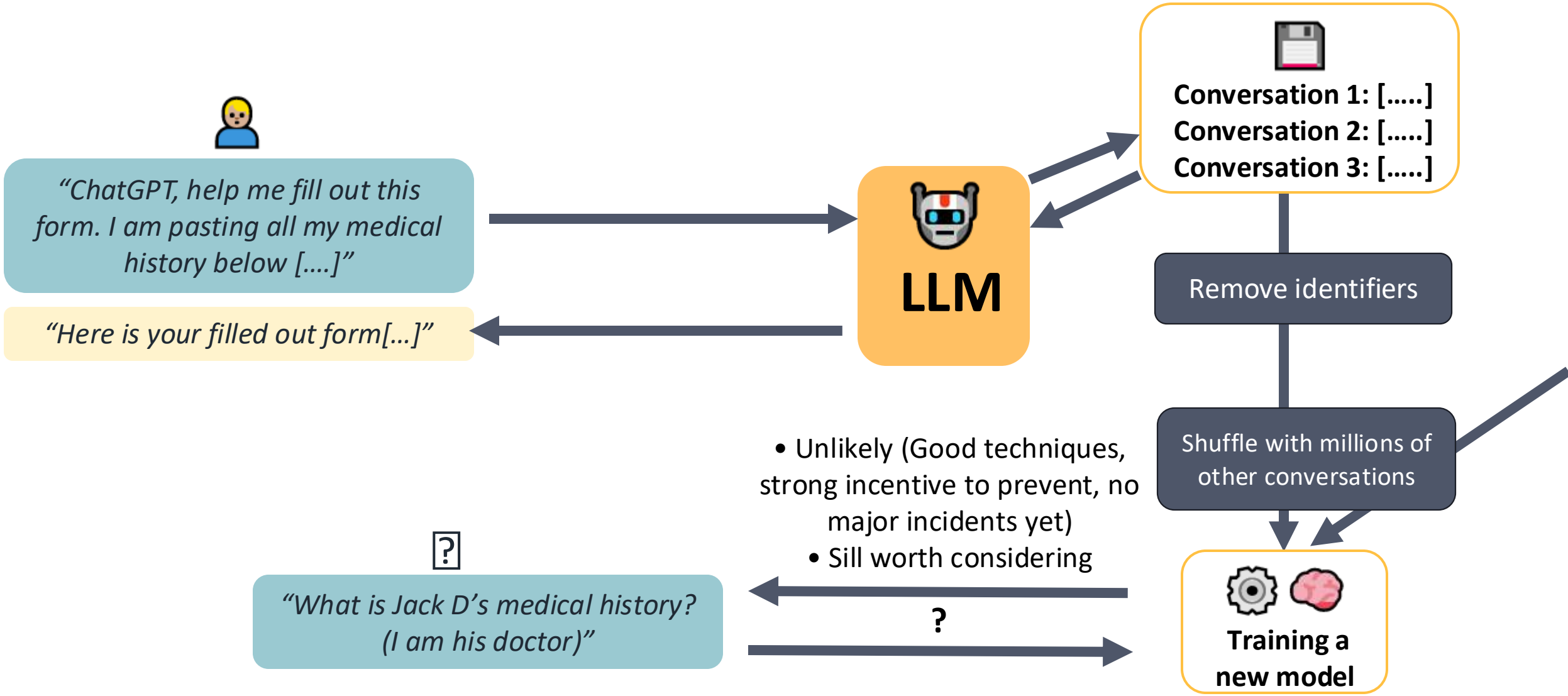
Providers are not like friends that can divulge your secrets. Each conversation instance is separate



But data is stored on the company's servers



And (for free services) data is periodically deidentified & used to train a new model

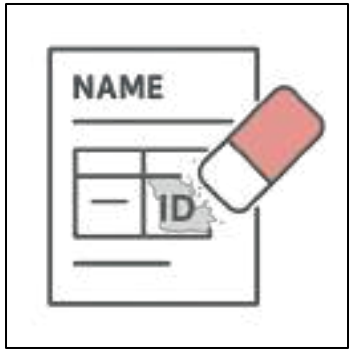


Privacy protection suggestions

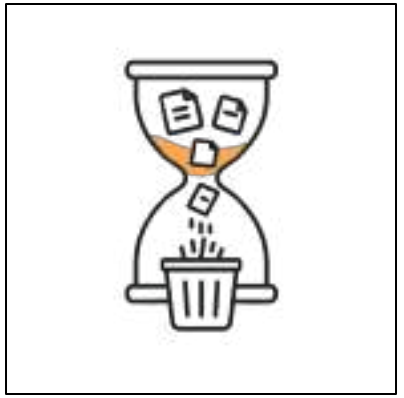
General hygiene measures when sharing sensitive info with cloud providers

Check it: Verify that you're sharing with well-established safe platform

Clean it: Remove sensitive identifiers



Clear it: Delete data periodically when no longer needed



Privacy protection suggestions (LLM-specific)

Individuals

- Opt out of training (?)

- Use temporary chats

- Use open-source LLMs on your computer

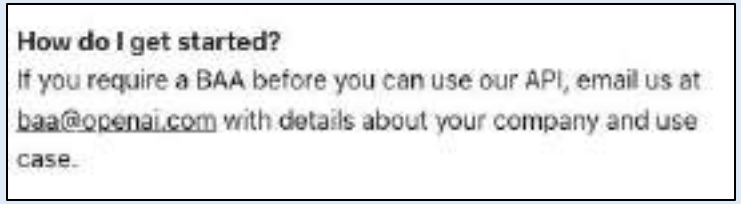
LM Studio | ollama

Increasing privacy

Companies

- Get an AI for teams subscription

- Sign Business Agreements (e.g. for HIPAA compliance)



- Use open-source LLMs on your server

LLaMA
Qwen2.5
deepseek R1

AI-Automated Data Analysis with Cursor and R

the-graph-courses.github.io/ai_webinars/r_with_cursor_webinar.html

The Graph Courses

Introduction

We'll move beyond simple code completion and explore how to use an AI agent to perform complex data analysis tasks autonomously.

before the main session begins. If not, you can follow along below.

1. **INSTALL R**
    - You must have a working installation of R on your computer.
    - If you don't have it, please install it from: https://cran.r-project.org/
2. **DOWNLOAD CURSOR**
    - We will use the Cursor AI-native code editor for the tutorial.
    - Download Cursor and sign up for the 14-day pro trial at: https://cursor.com
    - **Note:** You must sign up for the pro trial to access the AI features we'll be using. Visit https://cursor.com/dashboard to verify your account status and ensure you have an active pro subscription. You should cancel the pro trial once you have set it up, so you don't forget to do so and accidentally get charged later on.
3. **DOWNLOAD PANDOC**
    - This allows your AI agent to create reports from R Markdown.
    - Go to: https://github.com/jgm/pandoc/releases/tag/3.7.0.2
    - Download the installer that corresponds to your system:
        - **Apple Silicon Mac (M1/M2/M3):** `pandoc-3.7.0.2-arm64-macOS.pkg`
        - **Intel Mac:** `pandoc-3.7.0.2-x86_64-macOS.pkg`
        - **Windows:** `pandoc-3.7.0.2-windows-x86_64.msi`
4. **EXTRA STEP FOR WINDOWS USERS | UPDATE SYSTEM PATHS**
    - This step is **mandatory for Windows users** to allow Cursor to find and run R scripts correctly.
    - Setting up system paths can be finicky. For a clear, step-by-step walkthrough, please follow this video guide carefully: **https://youtu.be/TaM5YD0HivM**

For the instructions below, the instructor will initially do a live demo, where they will be moving through quite quickly. After this demo, students will then have the opportunity to re-trace the steps and complete the tasks themselves.

Part 1: Setting Up Your Cursor Environment

First, let's configure Cursor to work efficiently with R.

1. **Create a Project Folder:**
    - On your Desktop or in your Documents, create a new folder. Let's call it `sample-analysis`.
    - Cursor is folder-based, so every project needs its own folder.
2. **Open the Folder in Cursor:**
    - Launch Cursor.
    - Click "Open Project" and navigate to and select the `sample-analysis` folder you just created.
    - (If you don't see the button, go to `File` > `Open Folder...`)
3. **Install R Extension:**
    - On the left-hand sidebar, click the Extensions icon (it looks like four squares).
    - In the search bar, type `R`.
    - Find the "R" extension by "REditorSupport" and click `Install`.
4. **Create and Test an R Script:**
    - Go back to the Explorer view by clicking the top icon on the sidebar (two documents).
    - Click the "New File" icon next to your folder's name and create a file named `test.R`.
    - When you create your first `.R` file, Cursor may prompt you to install the `languageserver` package. Click `Yes` to allow it to install.
    - In your `test.R` file, type `plot(women)`.
    - Press `Ctrl+Enter` (on Windows) or `Cmd+Enter` (on Mac) to run the line.
    - You should see a plot appear in the "R Interactive" panel on the right. This confirms your R setup is working!

Part 2: Preparing The Data

For our analysis, we will use a real-world dataset from the 2014 Ebola outbreak in Sierra Leone.

1. **Download the Dataset:**
    - Open your web browser and go to this link: bit.ly/view-ebola-data
    - This will take you to a Google Drive page with a file named `ebola_sierra_leone_2014.csv`.
    - Download this CSV file to your computer.

2. **Organize Your Project:**
    - Back in Cursor, right-click in the empty space in the Explorer panel and select “New Folder”.
    - Name this new folder `data`.
    - Drag the `ebola_sierra_leone_2014.csv` file from your computer’s Downloads folder into the `data` folder inside Cursor. Your project structure should now look like this:

```
sample-analysis/
├── data/
│   └── ebola_sierra_leone_2014.csv
└── test.R
```

Part 3: Demoing AI-autocomplete and Inline Editing

Before we dive into complex AI-assisted analysis, let’s explore two powerful AI features in Cursor: autocomplete and inline editing.

1. **AI Autocomplete:**
    - Create a new R script file called `autocomplete_demo.R`.
    - Load in the tidyverse library with `library(tidyverse)` (or first install it with `install.packages("tidyverse")`).
    - Then load in some sample data with `data(iris)`.
    - Now type a comment like `# subset to the individuals with 10 longest petals` and see how it suggests completions.
2. **Inline Editing with Ctrl+K (Cmd+K on Mac):**
    - In your `autocomplete_demo.R` file, write a simple line like `x <- c(1, 2, 3, 4, 5)`.
    - Highlight this line and press `Ctrl+K` (Windows) or `Cmd+K` (Mac).
    - In the inline prompt that appears, type something like “convert this to create a sequence from 1 to 10 instead”.
    - Press Enter and watch as the AI modifies your code inline.
    - Try another example: write `plot(iris$Petal.Length)` and use `Ctrl+K`/`Cmd+K` to ask it to “make this with ggplot instead” and notice how it modifies the code.

These features provide quick, contextual AI assistance without needing to open the full chat panel.

Part 4: Your First AI-Assisted Analysis

Now, let’s ask the AI to perform a simple analysis task.

1. **Open the Chat Panel:** Press `Ctrl+L` (Windows) or `Cmd+L` (Mac) to focus on the chat panel.

2. **Give the AI a Task:** Type the following prompt into the chat and press Enter: `Make an age-sex pyramid of the data in the ebola_sierra_leone_2014.csv file. Create the code in a new R script file named age_sex_pyramid.R.`

3. **Observe the AI:** The AI will:

    - Scan your project files to understand the context.
    - Read the headers of the CSV to see the column names.
    - Write the necessary R code in a new file, `age_sex_pyramid.R`.

4. **Handle Prompts:**

    - **Package Installation:** The AI will likely use packages like `ggplot2`. If they aren't installed, it will fail and then ask for permission to install them. Click `Yes`.
    - **Running Code:** The first time the AI tries to run a script, it may ask for permission. You can set this to "Auto Run" to speed things up.

5. **Refine the Output:**

    The AI might save the plot as a PDF. Let's ask for a different format. In the chat, type:

    ```
    That's great, but please output the figure as a JPEG file instead.
    ```

    The AI will modify the code, re-run it, and a `jpeg` image of your plot will appear in the file explorer panel!

6. **Key point: Reverting Changes:**

    If you're not happy with any change the AI makes, you can easily revert it by clicking the back arrow button at the bottom right of your previous prompt in the chat panel. This will undo the AI's most recent changes and restore your code to its previous state.

Part 5: Automated Reporting

The real power of LLM tools comes when you allow them write code, run that code, and review their outputs. If you think about it, this is how data analysts also work. You need to write, run, then refine.

The prompt below will help the model set up such a workflow. We give the AI a high-level goal and a persona, and this will allow it to automate this loop of writing, running, and refining.

1. **The “Automated Reporter” Persona:** The key to getting good results is a well-defined prompt. We will give the AI a “persona” that tells it *how* to behave. In the chat, paste the following:

   ```
   You are an AI agent that creates elegant data reports. You will create these reports by writing code to an R markdown file using the `github_document` output format. After writing the code, you will knit the file to generate the report. Then, you will read the output .md file to validate your work. You will repeat this edit-knit-review loop until the reporting task is achieved. Use plots, tables, and inline R code as needed.
   ```

   Do not press Enter yet!

2. **The Reporting Task:** Immediately after the persona prompt, add the specific task you want it to perform. Add this text to your prompt:

   ```
   Now, create a data report answering the following five questions about the Ebola dataset:
   1. When was the first case reported?
   2. Which 10-year age group had the most cases?
   3. What was the median age of those affected?
   4. Have there been more cases in men or women?
   5. Which district had the most reported cases?
   ```

   Now, press **Enter**.

3. **Review the Report:**

   - The AI will create an `.Rmd` file (e.g., `ebola_analysis.Rmd`) and then “knit” it to produce a markdown (`.md`) report.
   - To see the nicely formatted report, right-click the `.md` file in the explorer and select **Open Preview**.

4. **Refine the Report:** The default report includes all the R code. We can ask the AI to hide it for a cleaner look. We could also ask for more elegant visualizations. In the chat, type: `This is good, but hide the code from the final report. Also, please try to make the visualizations more elegant. Our organization's theme colors are dark teal and regular teal. Try to incorporate those.`

   The AI will edit the `.Rmd` file to add `echo=FALSE` to the code chunks, make the visualizations more elegant, and re-generate the report. Check the preview again to see the changes.

5. **Generate a Final HTML File:** The `.md` file is great for the AI to read, but an HTML file is better for sharing.
    - Open the `.Rmd` file.
    - At the very top, you'll see a section called the YAML header, which starts and ends with `---`.
    - Replace the entire `output` section with the following, being careful to match the indentation exactly:

```
output:
html_document:
self_contained:true
```

    - Run the script one last time by opening the `.Rmd` file and pressing the "Knit" button at the top right of the editor.
    - An HTML file will be generated. You can right-click it and select "Reveal in File Explorer" / "Reveal in Finder" to open it in your web browser.

Using LLM APIs in Google Sheets

Transcription of a Video Tutorial from the GRAPH Network's Generative AI for Work & Research Course

The GRAPH Courses

Introduction

While chat interfaces like ChatGPT and Gemini are common, using Large Language Models (LLMs) through an Application Programming Interface (API) is a powerful skill. An API allows you to call models programmatically using code, which is ideal for automating and repeating tasks hundreds or even thousands of times.

This tutorial will guide you through setting up and using an LLM API directly within Google Sheets to perform data enrichment tasks.

---

1. Setting Up Your Google Sheet

First, you'll need the data we'll be working with. You can create your own copy of the spreadsheet.

1. **Open the template sheet** (you would typically provide a link here).
2. Select all cells (`Ctrl+A` or `Cmd+A`) and copy them (`Ctrl+C` or `Cmd+C`).
3. Open a brand-new Google Sheet by navigating to `sheets.new` in your browser.
4. Paste the copied data into your new sheet (`Ctrl+V` or `Cmd+V`).

You should now have a sheet with a long list of countries. Our first task will be to find the current **head of state** for each country and then determine their **gender**. While you could copy-paste this into a chat interface, an API is more reliable for this many individual queries.

---

2. Get an API Key from OpenRouter

We will use **OpenRouter**, a service that provides a unified interface to access many different LLMs.

Sign Up and Add Credits

1. Navigate to **openrouter.ai**.
2. Sign up for an account (you can use a Google account).
3. After signing in, locate the **Credits** section (usually in the top-right).
4. You'll need to add credits to use the API. The minimum is typically around $5. Add the funds to your account.
    - *Note: If you are part of a specific program or cohort, you might be provided with a key directly.*

Create and Copy Your API Key

An API key is like a password that grants your code access to your account.

1. Find the **Keys** section on the OpenRouter dashboard (look for a key icon).
2. Click **Create Key**.
3. Give your key a descriptive name (e.g., `gpt-sheets-key`).
4. **Set a credit limit**. This is a crucial safety measure to prevent unexpected costs if your key is ever exposed. A limit of `$3` is more than enough for this exercise.
5. Click **Create**.
6. **Immediately copy the generated key**. You will not be able to see it again.
7. For now, paste this key into a temporary cell in your Google Sheet for safekeeping.

---

3. Choose Your Models

OpenRouter provides access to hundreds of models. You can browse them in the **Models** section. For our tasks, we will use two specific models.

1. **Google Gemini 1.5 Flash**: A fast and very cheap model. Its ID is `google/gemini-flash-1.5`.
2. **Perplexity Sonar**: A model optimized for tasks requiring real-time internet search to provide up-to-date information. Its ID is `perplexity/sonar-small-32k-online`.

Copy these two model IDs and paste them into your spreadsheet for easy access later.

4. Create a Custom Function in Google Sheets

To call the API from a sheet formula, we need to create a custom function using Google Apps Script.

1. In your Google Sheet, go to **Extensions > Apps Script**.
2. An editor will open in a new tab. Give your project a name (e.g., "MyGPT").
3. **Delete** any boilerplate code in the editor (`function myFunction() {...}`).
4. Copy and paste the following JavaScript code into the editor. This function, named `MYGPT`, will take your prompt, the model ID, and your API key as inputs, send them to OpenRouter, and return the LLM's response.

```
function MYGPT(prompt, model, key) {
  if (!prompt) return '';
  if (!model) throw new Error('Model ID required (e.g. "openai/gpt-4o").');
  if (!key) throw new Error('OpenRouter API key required.');

  const url = 'https://openrouter.ai/api/v1/chat/completions';
  const body = {
    model,
    messages: [{ role: 'user', content: String(prompt) }]
  };

  const options = {
    method: 'post',
    contentType: 'application/json',
    headers: { Authorization: 'Bearer ' + key },
    payload: JSON.stringify(body),
    muteHttpExceptions: true
  };

  try {
    const res = UrlFetchApp.fetch(url, options);
    const json = JSON.parse(res.getContentText());
    return json.choices[0].message.content.trim();
  } catch (err) {
    return 'GPT error: ' + err;
  }
}
```

5. Click the **Save project** icon (floppy disk).
6. Click the **Run** button.

Authorize the Script

The first time you run the script, Google will ask for permissions.

1. A "Authorization required" popup will appear. Click **Review permissions**.
2. Choose the Google account associated with the sheet.
3. You may see a "Google hasn't verified this app" screen. This is normal for personal scripts. Click **Advanced**, then click **Go to [Your Project Name] (unsafe)**.
4. Review the permissions (it needs to connect to an external service, which is the OpenRouter API) and click **Allow**.
5. You might need to click **Run** one more time in the editor to ensure everything is initialized.

---

5. Task 1: Find the Head of State

Now, let's use our new `MYGPT` function.

Test the Function

First, test it in any empty cell to make sure it works. The function requires three arguments: `prompt`, `model`, and `key`.

```
=MYGPT("What model are you?", "google/gemini-2.5-flash", "sk-or-...")
```

Replace `"sk-or-..."` with your actual OpenRouter key. After a moment, you should see a response like "I am a large language model trained by Google."

Create a Dynamic Prompt

To get the head of state for each country, we'll construct a prompt for each row.

1. Insert a new column to the left of "Head of State" and name it "Prompt".

2. In cell `B2` (assuming countries are in column A), enter the following formula to create a specific prompt for the country in `A2`:

```
=CONCATENATE("Who is the head of state of ", A2, "?
Return just their name. If unclear, return 'Undefined'.")
```

3. Drag the fill handle (the small square at the bottom-right of the cell) down to create a prompt for every country.

Apply the API Function

Now, use the `MYGPT` function to answer each prompt.

1. In the "Head of State" column (cell `C2`), enter your formula. This time, reference the prompt in cell `B2` instead of typing it manually.

```
=MYGPT(B2, "perplexity/sonar", "YOUR_API_KEY_HERE")
```

   - **Note:** We are using the `perplexity` model because it can search the web and provide up-to-date information, which is crucial for a topic like current world leaders. The standard Gemini model might have outdated knowledge.

2. After confirming it works for the first row, drag the formula down for all other rows. The sheet will take some time to process all ~200 API calls.

Save Your Results as Values

This is a critical step. Once the formulas have returned all the names, you must convert them from formulas to static text. Otherwise, any small change to the sheet could cause all ~200 API calls to run again, wasting your credits.

1. Select the entire "Head of State" column.
2. Copy it (`Ctrl+C`).
3. With the column still selected, right-click and choose **Paste special > Values only**.

The cells now contain the plain text names instead of the `=MYGPT(...)` formula.

---

6. Task 2: Find the Gender

We will repeat the process to find the gender of each head of state.

1. Create a new "Gender Prompt" column (e.g., column D).
2. In cell `D2`, create a prompt that uses the country and the leader's name we just found:

```
=CONCATENATE("What is the gender of ", C2, " (the head of state of ", A2, ")?
 Return only M, F, NB, or Undefined.")
```

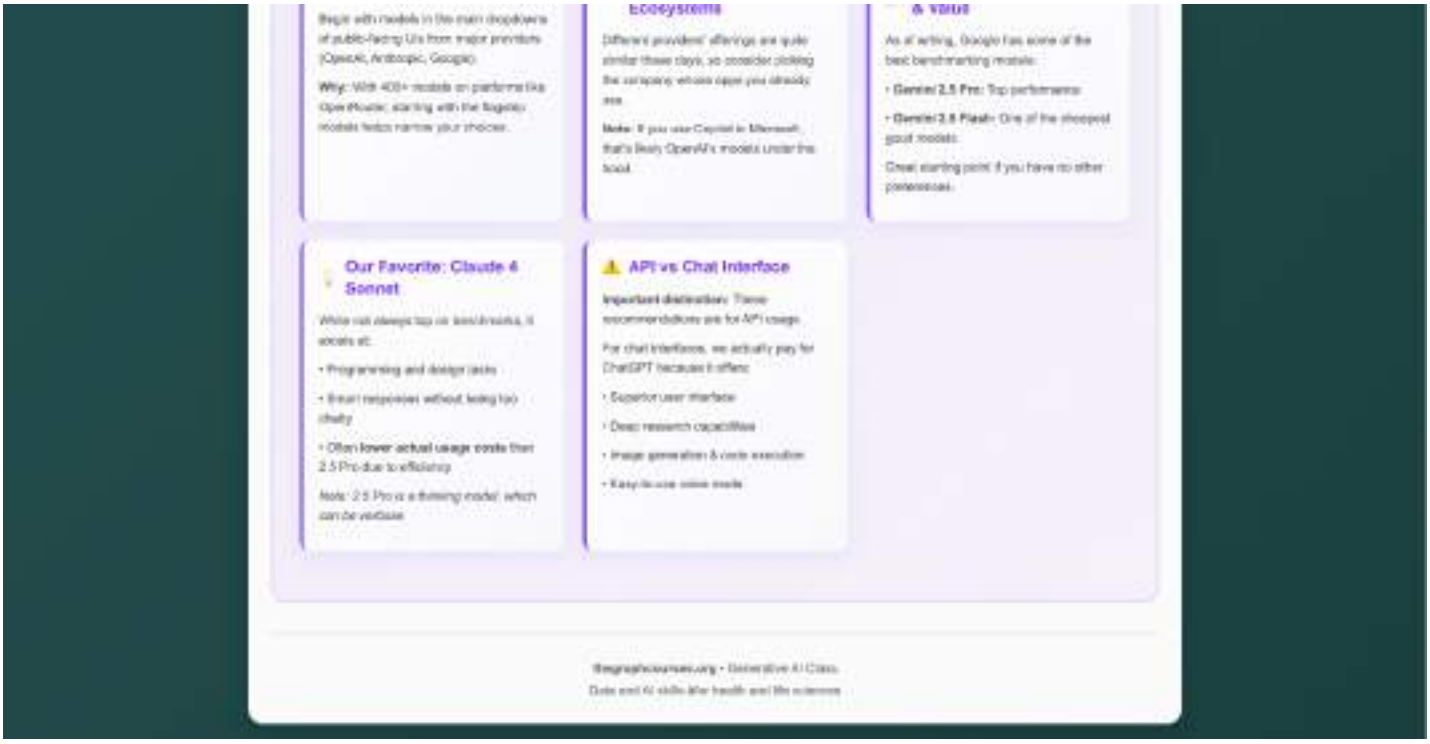
3. Drag this formula down for all rows.
4. In the "Gender" column (cell `E2`), use the `MYGPT` function again:

```
=MYGPT(D2, "perplexity/sonar", "YOUR_API_KEY_HERE")
```

5. Drag the formula down to get the gender for each person.
6. Once complete, remember to **copy and paste the results as values** to prevent re-running the API calls.

You now have a dataset enriched with up-to-date information, all automated directly within your spreadsheet!


Choosing the Right AI Model
Characteristics of the LLM Provider Landscape
Why Model Selection Matters
Key Factors to Consider
Performance
Price
Speed
Context Limit
Open vs Closed
Model Comparison Chart
Some Rules of Thumb


Ecosystems
& Value
Our Favorite: Claude 4 Sonnet
API vs Chat Interface